TDT4125 Algorithm Construction

Examination, August 8, 2022, 9:00–13:00

Academic contact	Magnus Lie Hetland
Support material code	A

Problems

1 Describe the difference between XP and FPT, in your own words. Do you see any parallels to approximation algorithms? Explain.

2 Your friend Lurvik has studied something he claims to be a matroid. His explanation is as follows:

> I have a set system consisting of a ground set E and a family of subsets **S** of E. I have managed to show that all bases, i.e., all maximal independent subsets, have the same cardinality.

Can Lurvik conclude that he has a matroid? What, if anything, is missing from his argument? Explain.

3 Where does the correctness proof for Christofides's algorithm break down, if we cannot assume the triangle inequality? Explain briefly.

4 What is the dual of the following linear program?

$$\begin{array}{ll} \text{maximize} & 3x_1 + 5x_2 \\ \text{subject to} & x_1 + 4x_2 \leqslant 10\,, \\ & 2x_1 - x_2 \geqslant 3 \\ \& & x_1\;,\quad x_2 \geqslant 0\,. \end{array}$$

Explain very briefly.

5 Your friend Gløgsund has managed to show that the following linear program always yields integer solutions:

$$
\begin{array}{lll}
\text{maximize} & \displaystyle\sum_{i=1}^{n}\Big(i\cdot\sum_{j=1}^{n}a_j x_{ij}\Big) & \\
\text{subject to} & \displaystyle\sum_{j=1}^{n} x_{ij} = 1 & (i = 1 \ldots n)\,, \\
 & \displaystyle\sum_{i=1}^{n} x_{ij} = 1 & (j = 1 \ldots n) \\
\& & x_{ij} \quad \geqslant 0 & (i, j = 1 \ldots n)\,.
\end{array}
$$

She remembers that the input is $a = [a_1, \ldots, a_n]$, but has forgotten what the program does. Which problem does it solve? Explain.

You may assume that all the values of a are distinct.

6 Your friend Smartnes has attempted to design a simplification of Kuhn-Munkres, which is still based on the primal–dual method, and that works for general graphs, not just bipartite ones. The input is a graph $G = (V, E)$, with cost $c : E \to \mathbf{R}_{\geqslant 0}$, and the goal is to find a perfect matching $M \subseteq E$ with as low a total cost as possible. Each edge $e \in E$ gets a binary primal variable $e.x$, where $e.x = 1$ means that $e \in M$, and $e.x = 0$ means that $e \notin M$. Smartnes describes his algorithm as follows:

> Start with $e.x = 0$ for all $e \in E$. Give each node $v \in V$ a dual variable $v.y = 0$. As long as there is some unmatched vertex u, increase $u.y$ until $u.y + v.y = c(u, v)$ for some edge $(u, v) \in E$, and let $(u, v).x = 1$.

Smartnes has studied the algorithm a bit, and thinks there may be nodes that will be matched with multiple others, and not just with one, as in an ordinary matching. He doesn't really mind, as long as all are matched with at least one other node, and this is done as cheaply as possible.

Discuss Smartnes's algorithm. Which connections do you see to the curriculum? Is it reasonable to claim this is a primal–dual algorithm? Will the solution have minimum cost, or will it be approximate? If so, what is the performance guarantee?

You may assume there is a perfect matching in the graph.

7 You are to send goods with as few trucks as possible. All trucks have the same capacity (in kg), and goods with different weights arrive on a conveyor belt. You fill one truck at a time, until no more goods can be added, and then you send it off and start filling the next one.

We can view this as an online algorithm, where you in an offline version would have all the goods available from the beginning (and would be free to distribute them between trucks as you wished). Show that the online algorithm is 2-competitive.

You may assume that no goods are heavier than the capacity of the trucks. For simplicity, you may also assume that the number of trucks you end up with is an even number.

8 Your friend Klokland has developed a randomized algorithm for finding a maximum-weight clique in a graph with vertex weights. That is, input is a graph $G = (V, E)$ and a weight function $w : V \to \mathbf{R}_{\geqslant 0}$, and you are to find a set $S \subseteq V$ with greatest possible weight sum $\sum_{v \in S} w(v)$, such that all nodes in S are adjacent to all the others in S. Klokland explains his algorithm as follows:

> Select a random permutation of the vertices, with uniform probability. In this order, keep adding the next vertex to the solution if it is adjacent to all vertices added so far.

Let δ be the minimum degree in the graph, i.e., the lowest number of neighbors (adjacent vertices) any vertex has. Find a performance guarantee for the resulting approximation, as a function of δ and $n = |V|$.

You are to find a guarantee for the *expected* performance.